

PROGRAMMING FOR PROBLEM SOLVING (BE01000121)



Prepared By,

Prof. Shraddha Modi

Computer Engineering

LDCE, Ahmedabad

CHAPTER-5

ARRAY & STRING

Prepared By,
Prof. Shraddha Modi
Computer Engineering
LDCE, Ahmedabad

ROADMAP

- ❑ Concepts of array,
- ❑ One and two dimensional arrays,
- ❑ Declaration and initialization of arrays,
- ❑ String, string storage,
- ❑ Built-in string functions

INTRODUCTION

- Array is a fixed-size sequenced collection of elements of the same data type. (Derived data type)
- Array is useful concept for representing a group of similar value under a single name.
- Whenever a particular value from a group is needed, we can access it by name of array and the position of the value in the group.
- Example:- group of marks of ten students,
group of salary of 100 employee etc.

INTRODUCTION

```
int rollno[100];
```



Fixed Size

Here, the size of an array is 100 (fixed) to store rollno

Sequential

It is indexed to 0 to 99 in sequence

Same Data type

All the elements (0-99) will be integer variables

Single Name

All the elements (0-99) will be referred as a common name rollno

INTRODUCTION

- Position is called a index or superscript. Base index = 0.
- It allocates sequential memory locations. Individual values are called as elements.
- **Types of Arrays:** We can use arrays to represent not only simple lists of values but also tables of data in two or three or more dimensions.
 1. One – dimensional arrays
 2. Two – dimensional arrays
 3. Multidimensional arrays

ONE DIMENSIONAL ARRAY

- Declaration of One-Dimensional Arrays :

- The general form of array declaration is

`<datatype> <array_name>[sizeofarray];`

- The datatype specifies the type of element that will be contained in the array, such as int, float, or char.
- The size indicates the maximum number of elements that can be stored inside the array.
- The size of array should be a constant value.

ONE DIMENSIONAL ARRAY

- Examples:

float height[50];

- Declares the height to be an array containing 50 real elements. Any index 0 to 49 are valid.

int group[10];

- Declares the group as an array to contain a maximum of 10 integer numbers.

char name[10];

- Declares the name as a character array (string) variable that can hold a maximum of 10 characters.

ONE DIMENSIONAL ARRAY

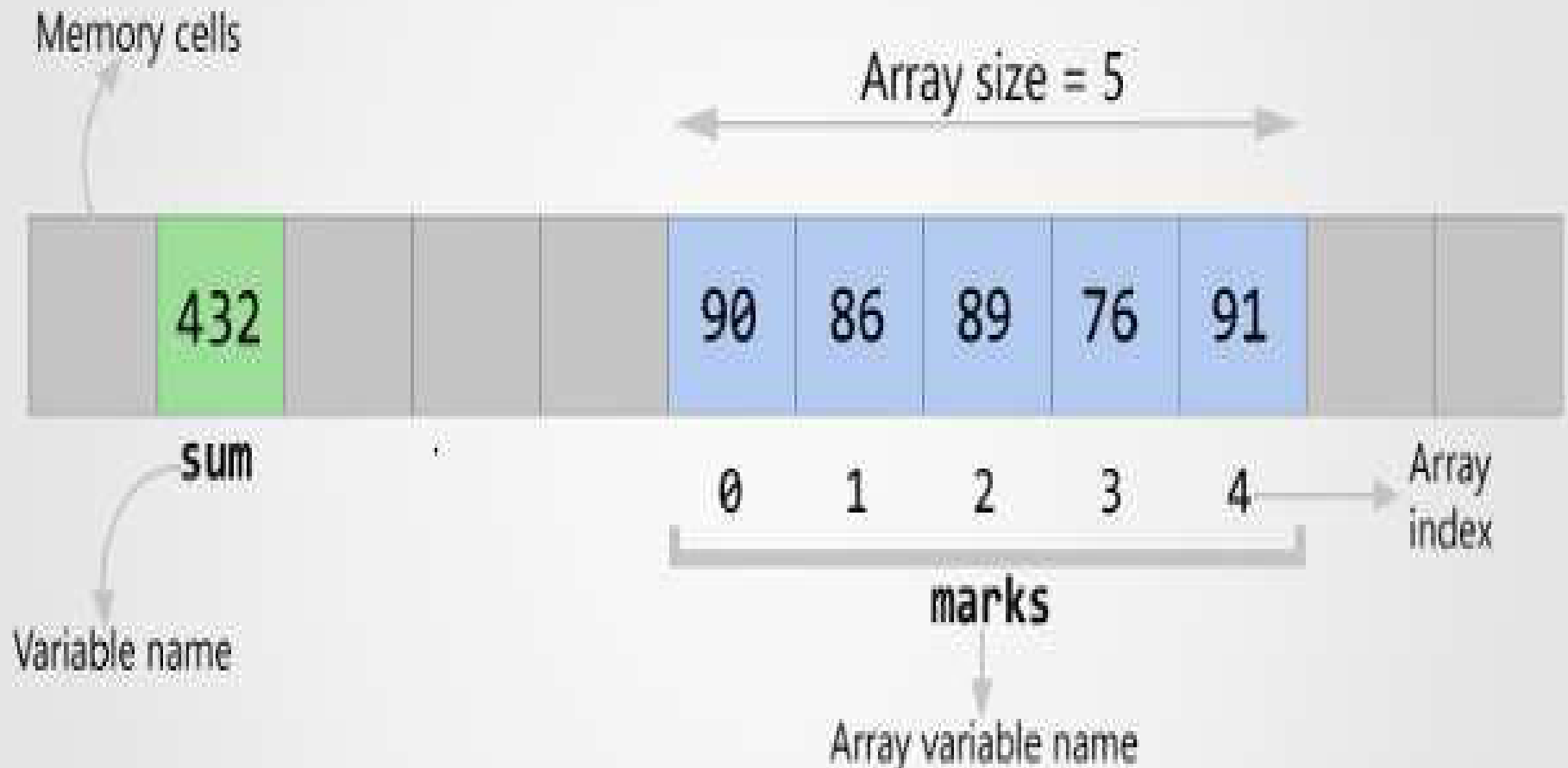
- Declaring, initializing and accessing single integer variable:

```
int mark=90;           //variable mark is initialized with value 90
printf("%d",mark);    //mark value printed
```

- Declaring, initializing and accessing integer array variable

```
int mark[5]={85,75,76,55,45}; //mark is initialized with 5 values
printf("%d",mark[0]); //prints 85
printf("%d",mark[1]); //prints 75
printf("%d",mark[2]); //prints 65
printf("%d",mark[3]); //prints 55
printf("%d",mark[4]); //prints 45
```

ONE DIMENSIONAL ARRAY



ONE DIMENSIONAL ARRAY

INITIALIZING THE ARRAY DURING THE DECLARATION

```
int array[6] = { 2,5,1,9,7,12 };
```

```
int array[3];
```

```
array[0]=5;  
array[1]=7;  
array[2]=2;
```

ONE DIMENSIONAL ARRAY

- Valid Statements :

1. `a = number[0] + 10;`

2. `number[4] = number[0] + number[2];`

3. `number[2] = x[5] + y[10];`

4. `value[6] = number[i] * 3;`

ONE DIMENSIONAL ARRAY

```
int vector[5];
```

vector[0]	100
vector[1]	104
vector[2]	108
vector[3]	112
vector[4]	116

...

...

...

...

...

ONE DIMENSIONAL ARRAY

ADDRESS CALCULATION

Actual Address of the 1st element of the array is known as

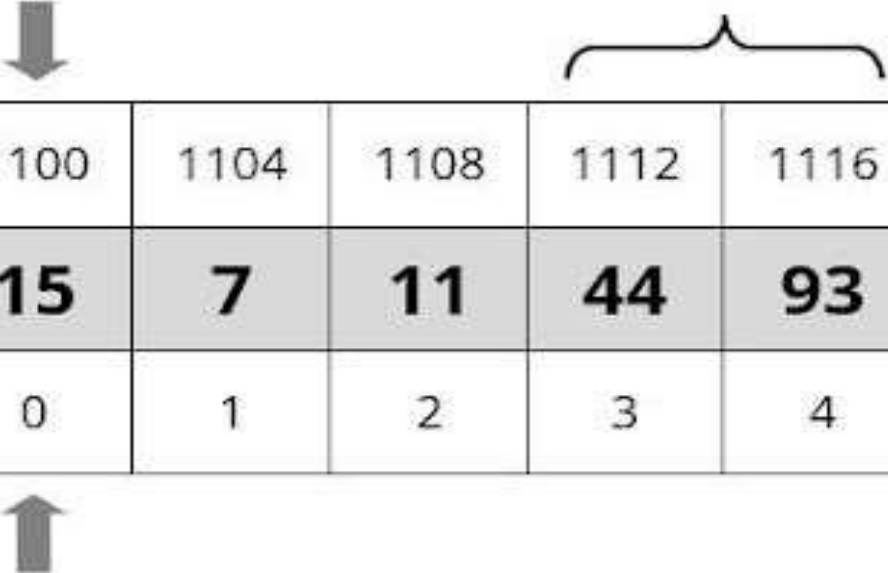
Base Address (B)

Here it is 1100

Memory space acquired by every element in the Array is called

Width (W)

Here it is 4 bytes



Actual Address in the Memory	1100	1104	1108	1112	1116	1120
Elements	15	7	11	44	93	20
Address with respect to the Array (Subscript)	0	1	2	3	4	5

Lower Limit/Bound of Subscript (**LB**)

ONE DIMENSIONAL ARRAY

READ(SCAN) ARRAY ELEMENTS

```
void main()
{
    int mark[5];
    printf("Enter array element=");
    scanf("%d",&mark[0]);
    printf("Enter array element=");
    scanf("%d",&mark[1]);
    printf("Enter array element=");
    scanf("%d",&mark[2]);
    printf("Enter array element=");
    scanf("%d",&mark[3]);
    printf("Enter array element=");
    scanf("%d",&mark[4]);

    printf("%d",mark[0]);
    printf("%d",mark[1]);
    printf("%d",mark[2]);
    printf("%d",mark[3]);
    printf("%d",mark[4]);
}
```

```
void main()
{
    int mark[5],i;
    for(i=0;i<5;i++)
    {
        printf("Enter array element=");
        scanf("%d",&mark[i]);
    }
    for(i=0;i<5;i++)
    {
        printf("%d",mark[i]);
    }
}
```

ONE DIMENSIONAL ARRAY

```
void main ( )  
{  
int a[10];  
char c[10];  
float b[10];  
printf("the type int requires %d bytes", sizeof(int));  
printf(" \n The type char requires %d bytes", sizeof(char));  
printf(" \n The type float requires %d bytes", sizeof(float));  
printf(" \n %d memory locations are reserved for ten int elements", sizeof(a));  
printf("\n %d memory locations are reserved for ten char elements",sizeof(c));  
printf (" \n %d memory locations are reserved for ten float elements",sizeof(b));  
}
```


ONE DIMENSIONAL ARRAY

Output:

The type int requires 2 bytes

The type char requires 1 bytes

The type float requires 4 bytes

20 memory locations are reserved for ten int elements

10 memory locations are reserved for ten char elements

40 memory locations are reserved for ten float elements

ONE DIMENSIONAL ARRAY

Write a program to display character array with their address.

```
#include<stdio.h>
void main( )
{
char name[10] = {'A', 'R', 'R', 'A', 'Y'};
int i = 0;

printf("\n character memory location \n");
while(name[i] != '\0')
{
printf("\n %c %u", name[i], &name[i]);
i++;
}
}
```

character memory location

A 351970

R 351971

R 351972

A 351973

Y 351974

ONE DIMENSIONAL ARRAY

PROGRAM EXERCISE1

- Develop a program to count number of positive or negative number from an array of 10 numbers.

```
void main(){
    int num[10],i,pos,neg;
    pos = 0;
    neg = 0;
    for(i=0;i<10;i++)
    {
        printf("Enter array element=");
        scanf("%d",&num[i]);
    }
    for(i=0;i<10;i++)
    {
        if(num[i]>0)
            pos=pos+1;
        else
            neg=neg+1;
    }
    printf("Positive=%d,Negative=%d",pos,neg);
}
```

Output

```
Enter array element=1
Enter array element=2
Enter array element=3
Enter array element=4
Enter array element=5
Enter array element=-1
Enter array element=-2
Enter array element=3
Enter array element=4
Enter array element=5
Positive=8,Negative=2
```

ONE DIMENSIONAL ARRAY

PROGRAM EXERCISE1

- Write a program to find out the largest and smallest element in an array.

```
#include<stdio.h>
void main( )
{
    int i,n,a[50], large, small;
    printf("Enter size of array:");
    scanf("%d", &n);
    printf("\n Array elements are \n");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    large = a[0];
    small = a[0];
    for(i=1; i<n; i++)
    {
        if(a[ i ] > large)
            large = a[ i ];
        else if(a[ i ] < small)
            small = a[ i ];
    }
    printf("\n Largest element in array is %d \n", large);
    printf(" \n smallest element in array is %d \n", small);
}
```

Enter size of array:5

Array elements are

56

12

4

6

1

Largest element in array is 56

smallest element in array is 1

FUNCTION AND ARRAY

```
#include<stdio.h>

int largest(int a[], int n);

void main()
{
    int value[4] = {23, 45, 32, 11};
    printf("%d", largest(value , 4));
}

int largest(int a[], int n)
{
    int i, max;
    max = a[0];
    for(i = 1; i<n; i++)
    {
        if(max<a[i])
        {
            max = a[i];
        }
    }
    return(max);
}
```

TWO DIMENSIONAL ARRAY

Syntax

```
data-type variable-name[x][y];
```

Declaration

```
int data[3][3]; //This array can hold 9 elements
```

- A two dimensional array can be seen as a table with '**x**' rows and '**y**' columns.
- The row number ranges from 0 to (x-1) and column number ranges from 0 to (y-1).

TWO DIMENSIONAL ARRAY

- For example,

```
int data[3][3];
```

	Column-0	Column-1	Column-2
Row-0	<code>data[0][0]</code>	<code>data[0][1]</code>	<code>data[0][2]</code>
Row-1	<code>data[1][0]</code>	<code>data[1][1]</code>	<code>data[1][2]</code>
Row-2	<code>data[2][0]</code>	<code>data[2][1]</code>	<code>data[2][2]</code>

TWO DIMENSIONAL ARRAY INITIALIZATION

- Valid initialization:

```
1. int table[2][3] = {0,0,0,1,1,1};
```

```
2. int table[2][3] = { {0,0,0}, {1,1,1} };
```

```
3. int table[2][3] = {  
    {0,0,0},  
    {1,1,1}  
}
```


TWO DIMENSIONAL ARRAY INITIALIZATION

- Valid initialization:

4. `int table [2] [3] = {`

`{1,1},`

`{2}`

`};`

1	1	0
2	0	0

5. When all the elements are to be initialized to zero, we can write:

`int table[3][5] = { {0}, {0}, {0} };`

OR

`int m[3][5] = { 0, 0 }; OR      int m[3][5] = { 0 };`

TWO DIMENSIONAL ARRAY

- Program to read the matrix elements and display the same in matrix form.

```
void main(){
    int data[3][3],i,j;
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("Enter array element=");
            scanf("%d",&data[i][j]);
        }
    }
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("%d",data[i][j]);
        }
        printf("\n");
    }
}
```

Output

```
Enter array element=1
Enter array element=2
Enter array element=3
Enter array element=4
Enter array element=5
Enter array element=6
Enter array element=7
Enter array element=8
Enter array element=9
123
456
789
```


TWO DIMENSIONAL ARRAY

- Program to do Summation in Two Dimensional Array in C.

```
#include <stdio.h>

int main() {
    int r, c, a[100][100], b[100][100], sum[100][100], i, j;
    printf("Enter the number of rows (between 1 and 100): ");
    scanf("%d", &r);
    printf("Enter the number of columns (between 1 and 100): ");
    scanf("%d", &c);

    printf("\nEnter elements of 1st matrix:\n");
    for (i = 0; i < r; ++i)
        for (j = 0; j < c; ++j) {
            printf("Enter element a%d%d: ", i + 1, j + 1);
            scanf("%d", &a[i][j]);
        }
}
```

Program
continues

TWO DIMENSIONAL ARRAY

- Program to do Summation in Two Dimensional Array in C.

```
printf("Enter elements of 2nd matrix:\n");
for (i = 0; i < r; ++i)
    for (j = 0; j < c; ++j) {
        printf("Enter element b%d%d: ", i + 1, j + 1);
        scanf("%d", &b[i][j]);
    }

// adding two matrices
for (i = 0; i < r; ++i)
    for (j = 0; j < c; ++j) {
        sum[i][j] = a[i][j] + b[i][j];
    }
```


TWO DIMENSIONAL ARRAY

- Program to do Summation in Two Dimensional Array in C.

```
// printing the result
printf("\nSum of two matrices: \n");
for (i = 0; i < r; ++i)
    for (j = 0; j < c; ++j) {
        printf("%d    ", sum[i][j]);
        if (j == c - 1) {
            printf("\n\n");
        }
    }

return 0;
}
```

TWO DIMENSIONAL ARRAY

- Program to do Summation in Two Dimensional Array in C.

- Output:

```
Enter the number of rows (between 1 and 100): 2
Enter the number of columns (between 1 and 100): 2
```

```
Enter elements of 1st matrix:
```

```
Enter element a11: 1
```

```
Enter element a12: 2
```

```
Enter element a21: 3
```

```
Enter element a22: 4
```

```
Enter elements of 2nd matrix:
```

```
Enter element b11: 4
```

```
Enter element b12: 3
```

```
Enter element b21: 2
```

```
Enter element b22: 1
```

```
Sum of two matrices:
```

```
5  5
```

```
5  5
```


ARRAY - QUIZ

Consider the following array declaration in the 'C' language:

```
int array 1[] = {2, 3};
```

```
int array2[3]={9};
```

What will be the output of the following print statement?

```
printf("%d, %d", array1 [1], array2 [2]);
```

1. 2, 0

2. 3, 0



3. 2, 9

4. 3, 9

ARRAY - QUIZ

What will be the output of the following statement?

```
printf("%d", 2[array]);
```

1. 4

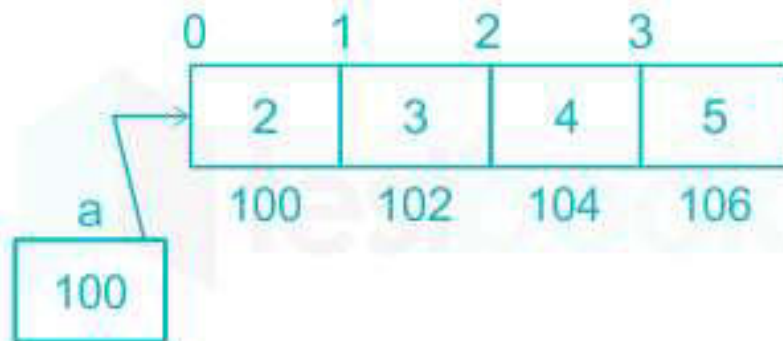


2. 3

3. 2

4. 5

int array[] = {2, 3, 4, 5}; This array storage be like,



The above array at **index 1** can be accessed by, `a[i]`, `i[a]`, `*(a+i)` or `*(i+a)` all the above representations all are equal and gives the *i*th index value.

ARRAY – QUIZ

GUESS THE OUTPUT

```
int main()
{
    int arr[] = { 2, 3, 4, 5 };
    printf("%d ",arr[2]);
    printf("%d ",2[arr]);
    printf("%d ",*(2+arr));
    printf("%d ",*(arr+2));
    return 0;
}
```

O/P : 4 4 4 4

ARRAY - QUIZ

If integer requires two bytes space, then what will be the size of the following 'C' array?

```
int array[3][4]=(0);
```

1. 24 bytes



2. 12 bytes

3. 7 bytes

4. 14 bytes

ARRAY – QUIZ

GUESS THE OUTPUT

```
1.  #include <stdio.h>
2.  void main()
3.  {
4.      int a[2][3] = {1, 2, 3, 4, 5};
5.      int i = 0, j = 0;
6.      for (i = 0; i < 2; i++)
7.          for (j = 0; j < 3; j++)
8.              printf("%d", a[i][j]);
9.  }
```

- a) 1 2 3 4 5 0 ✓
- b) 1 2 3 4 5 junk
- c) 1 2 3 4 5 5
- d) Run time error

ARRAY – QUIZ

GUESS THE OUTPUT

```
1.    #include <stdio.h>
2.    void main()
3.    {
4.        int a[2][3] = {1, 2, 3, , 4, 5};
5.        int i = 0, j = 0;
6.        for (i = 0; i < 2; i++)
7.            for (j = 0; j < 3; j++)
8.                printf("%d", a[i][j]);
9.    }
```

a) 1 2 3 junk 4 5

b) Compile time error



c) 1 2 3 0 4 5

d) 1 2 3 3 4 5

ARRAY - QUIZ

Which of the following is not a valid declaration of arrays?

A. `int marks[4] = { 67, 87, 56, 77 }`

B. `float area[5] = { 23.4, 6.8, 5.5 }`

C. `int marks[] = { 67, 87, 56, 77, 59 }`

D. `int marks[4] = { 67, 87, 56, 77, 59 }`



ARRAY - QUIZ

Q. What will be the output of given code?

```
main()
{
    int arr[3] = {1, 2};
    printf("%d", &arr[2]);
}
```

☐

A. error

☐

B. garbage value

☐

C. 1

☐

D. 2

ARRAY - QUIZ

What is the correct syntax to access the fourth element of an array named `arr`?

- A) `arr[4]`
- B) `arr(4)`
- C) `arr{4}`
- D) `arr[3]`



STRING

- A string is a sequence of characters that are terminated with a null or '\0' character.

- *Syntax :*

- `char String_Variable_name [ SIZE ] ;`

- *Examples :*

- `char city[30];`
 - `char name[20];`
 - `char message[50];`

String constant → "A"
Character constant → 'A'

A

\0

A

STRING INITIALIZATION

1. `char str[] = "Computer";`
2. `char str[50] = "Computer";`
3. `char str[] = {'C','o','m','p','u','t','e','r','\0'};`
4. `char str[10] = {'C','o','m','p','u','t','e','r','\0'};`
5. Note: If the array cannot accommodate the entire string, it only takes characters based on its space.

For example:

```
char str[3] = "Scaler";  
printf("%s",str);
```

```
//char *p = "Scaler";  
// printf("%s",p);
```

STRING INITIALIZATION

6. Character arrays cannot be assigned a string literal with the '=' operator once they have been declared.

```
char str[100];  
str = "String.";
```

This will cause a compilation error since the assignment operations are not supported once strings are declared.

STRING

READ STRING FROM THE USER

- `scanf()`, reads a sequence of characters until it encounters whitespace (i.e., space, newline, tab, etc.)

```
void main()
{
    char name[25];
    scanf("%s", name);
    printf("Name = %s \n", name);
}
```

Output1:

Hello
Name = Hello

Output2:

Hello Hi
Name = Hello

STRING

READ STRING FROM THE USER

- **Note -**

1. The format specifier used to input and output strings in C is %s.
2. You might notice that generally, the variable's name is preceded by a & operator with *scanf()*.

This is not the case here, as the character array is a pointer that points to the address of the first character of the array.

Hence the address operator (&) doesn't need to be used.

STRING

READ STRING FROM THE USER

□ gets() and puts ()

- gets() function reads a line from stdin into the buffer pointed to by s until either a terminating newline or EOF (End of File) occurs.
- puts() function writes the string 's' and 'a' trailing newline to stdout.

STRING

READ STRING FROM THE USER

□ gets() and puts ()

```
#include <stdio.h>
void main( )
{
    /*Character array of length 100*/
    char str[100];
    printf("Enter a string: ");
    /* Take a string as input */

    gets(str);
    /* Display the string */

    printf("Entered string is: ");
    puts(str);
    printf("Entered string is = %s",str);
}
```

Enter a string: india is great

Entered string is: india is great

Entered string is: india is great

STRING

FEATURES OF THE %S SPECIFICATIONS.

1. When the **field width** is less than the length of the string, the entire string is printed.
2. The **integer value on the right side** of the decimal point specifies the number of characters to be printed.
3. When the number of characters to be printed is specified as **zero**, nothing is printed.
4. The **minus sign** in the specification causes the string to be printed left-justified.
5. The specification **% .ns** prints the first n characters of the string.

STRING

FEATURES OF THE %S SPECIFICATIONS.

```
void main()
{
    char country[15] = "United Kingdom";
    printf("----- \n");
    printf("1. %15s\n", country);
    printf("2. %5s\n", country);
    printf("3. %15.6s\n", country);
    printf("4. %-15.6s\n", country);
    printf("5. %15.0s\n", country);
    printf("6. %.3s\n", country);
    printf("7. %s\n", country);
    printf("----- \n");
}
```

```
-----
1. United Kingdom
2. United Kingdom
3. United
4. United
5.
6. Uni
7. United Kingdom
-----
```


FUNCTION AND STRING

```
#include<stdio.h>                void display(char sname[])  
  
void display(char sname[]); {  
  
void main()                        printf(" %s", sname);  
  
{                                }  
  
char name[6] = "Comp-R";  
  
display(name);  
  
}
```


STRING

INBUILT FUNCTIONS

- **strlen():** This function can be used to find a string's length.

```
#include <stdio.h>
#include <string.h>

int main() {
    int a;
    char str[] = "Hello World";
    a = strlen(str);
    printf("The length of the string is: %d", a);
    return 0;
}
```

The length of the string is: 11

STRING

INBUILT FUNCTIONS

- Guess the output:

```
#include <stdio.h>
#include <string.h>

int main() {
    char alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    printf("Length is: %d\n", strlen(alphabet));
    printf("Size is: %d\n", sizeof(alphabet));
    return 0;
}
```

Length is: 26

Size is: 27

STRING

INBUILT FUNCTIONS

- Guess the output:

```
#include <stdio.h>
#include <string.h>

int main() {
    char alphabet[50] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    printf("Length is: %d\n", strlen(alphabet));
    printf("Size is: %d\n", sizeof(alphabet));
    return 0;
}
```

```
Length is: 26
Size is: 50
```


STRING

INBUILT FUNCTIONS

- **strcpy():** This function is used to copy one string to another.

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello World";
    char dest[20];
    strcpy(dest, src);
    printf("The copied string is: %s\n", dest);
    return 0;
}
```

The copied string is: Hello World

STRING

INBUILT FUNCTIONS

- **strcat():** This function is used to concatenate two strings.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[] = "Hello";
    char str2[] = "World";
    strcat(str1, str2);
    printf("The concatenated string is: %s\n", str1);
    return 0;
}
```

The concatenated string is: HelloWorld

STRING

INBUILT FUNCTIONS

- Guess the output

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[10] = "Hello ";
    char str2[] = "World!";

    strcat(str1, str2);
    printf("%s", str1);

    return 0;
}
```

```
*** buffer overflow detected ***: terminated
```


STRING

INBUILT FUNCTIONS

- **strcmp():** This function is used to compare two strings.

```
#include<stdio.h>
#include<string.h>

int main() {
    char string1[] = "programming";
    char string2[] = "programmer";

    int value = strcmp(string1, string2);
```

```
    if (value == 0)
        printf("Strings are equal");
    else
        printf("Strings are unequal");
```

```
    printf("\nValue returned by strcmp is: %d", value);
    return 0;
```

```
}
```

Strings are unequal

Value returned by strcmp is: 4

STRING

INBUILT FUNCTIONS

- **strcmp():** This function is used to compare two strings.

```
#include<stdio.h>
#include<string.h>

int main() {
    char string1[] = "programming";
    char string2[] = "programming";

    int value = strcmp(string1, string2);

    if (value == 0)
        printf("Strings are equal");
    else
        printf("Strings are unequal");

    printf("\nValue returned by strcmp is: %d", value);
    return 0;
}
```

Strings are equal

Value returned by strcmp is: 0

STRING

MORE INBUILT FUNCTIONS

- `char s1[]="Their", s2[]="There";`

`strchr(s1,c)`

Returns a pointer to the first occurrence of a given character in the string s1.
`printf("%s",strchr(s1,'i'));`
Output : ir

`strstr(s1,s2)`

Returns a pointer to the first occurrence of a given string s2 in string s1.
`printf("%s",strstr(s1,"he"));`
Output : heir

`strrev(s1)`

Reverses given string.
`strrev(s1);`
makes string s1 to "rieht"

STRING

MORE INBUILT FUNCTIONS

- `char s1[]="Their", s2[]="There";`

`strlwr(s1)`

Converts string s1 to lower case.
`printf("%s",strlwr(s1));`
Output : their

`strupr(s1)`

Converts string s1 to upper case.
`printf("%s",strupr(s1));`
Output : THEIR

`strncpy(s1,s2,n)`

Copies first n character of string s2 to string s1
`s1=""`; `s2="There"`;
`strncpy(s1,s2,2);`
`printf("%s",s1);`
Output : Th

STRING

MORE INBUILT FUNCTIONS

- `char s1[]="Their", s2[]="There";`

`strncat(s1,s2,n)`

Appends first n character of string s2 at the end of string s1.

```
strncat(s1,s2,2);  
printf("%s", s1);  
Output : TheirTh
```

`strncmp(s1,s2,n)`

Compares first n character of string s1 and s2 and returns similar result as `strcmp()` function.

```
printf("%d",strcmp(s1,s2,3));  
Output : 0
```

STRING - QUIZ

Find the number of elements in the following array.

```
char si [] = "India is great";
```

1. 3

2. 5

3. 15



4. 12

ARRAY - QUIZ

Q. What will be the output of given code

```
main()
{
    char ch[20];
    ch = "Study Tonight";
    printf("%s", ch);
}
```

☐ A. error



☐ B. Study Tonight

☐ C. Study

☐ D. Tonight

STRING

INBUILT FUNCTIONS

Guess the output

```
#include <stdio.h>

int main() {
    char str[10] = "World";
    str[0] = 'H';
    printf("%s\n", str);
    return 0;
}
```

Horld

STRING

INBUILT FUNCTIONS

Guess the output

```
#include <stdio.h>
int main()
{
    printf("%s\n", 4+"Hello world");
    printf("%s\n", "Hello world"+4);
    return 0;
}
```

```
o world
o world
```


STRING

INBUILT FUNCTIONS

Guess the output

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
```

```
    char str[]="Hello";
```

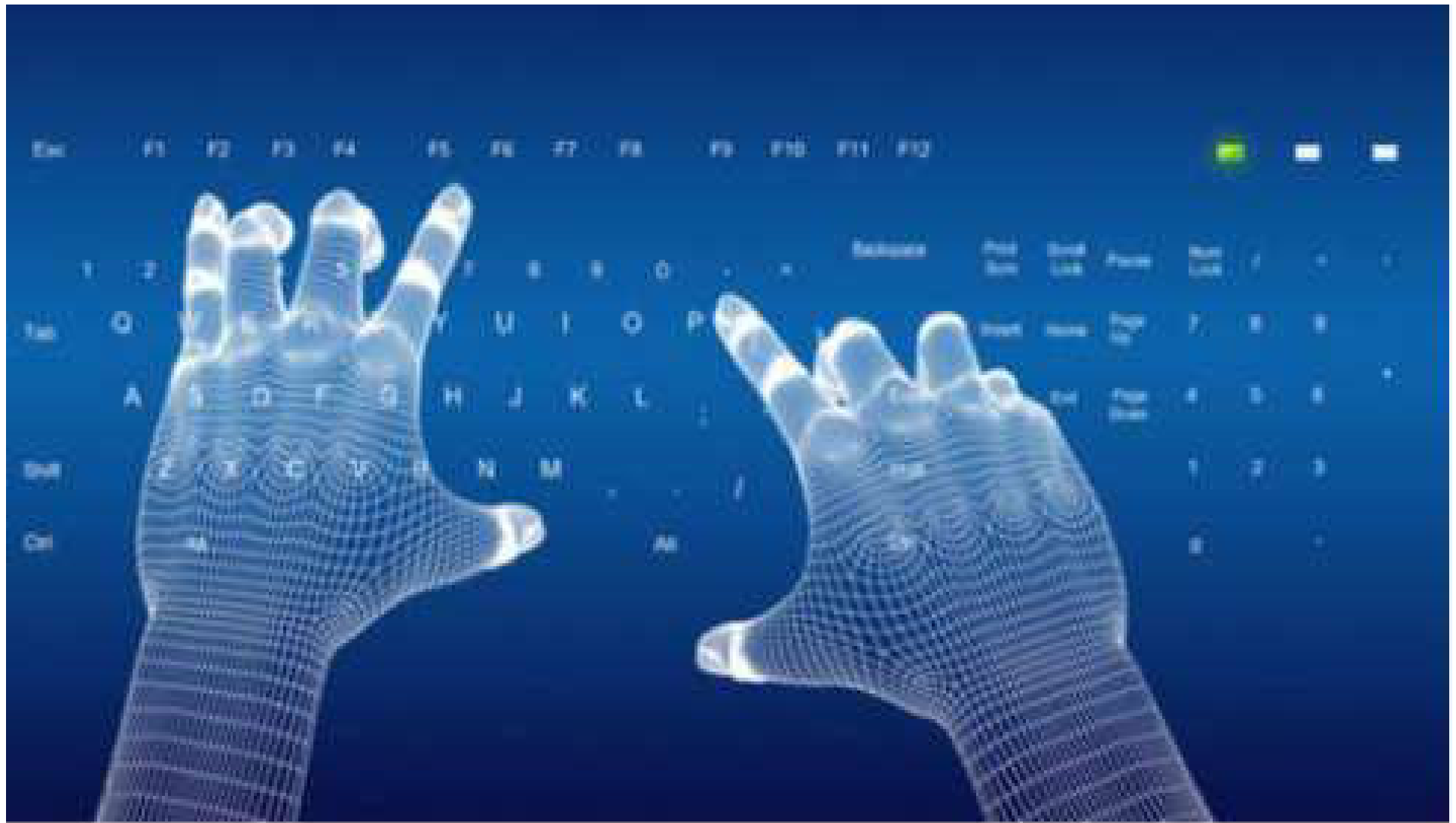
```
    str[strlen(str)+1]='#';
    printf("str= %s\n",str);
```

```
    return 0;
```

```
}
```

```
str= Hello
```

```
*** stack smashing detected ***: terminated
```



Thank You !